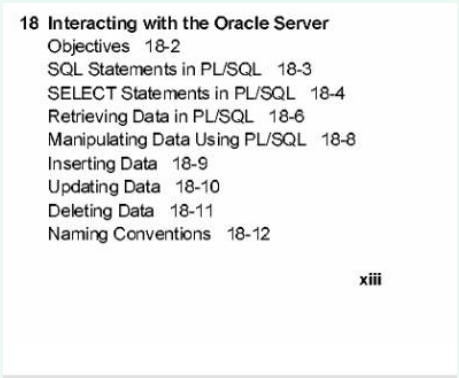
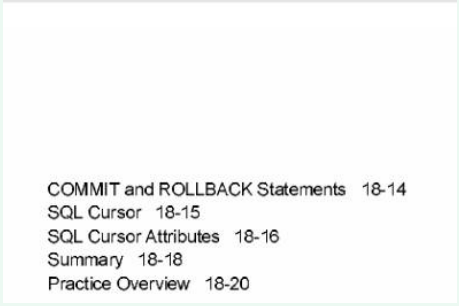
PL/SQL Oracle Interaction


18 Interacting with the Oracle Server
Objectives 18-2
SQL Statements in PL/SQL 18-3
SELECT Statements in PL/SQL 18-4
Retrieving Data in PL/SQL 18-6
Manipulating Data Using PL/SQL 18-8
Inserting Data 18-9
Updating Data 18-10
Deleting Data 18-11
Naming Conventions 18-12
xiii


COMMIT and ROLLBACK Statements 18-14
SQL Cursor 18-15
SQL Cursor Attributes 18-16
Summary 18-18
Practice Overview 18-20

go through the contents and explain everything about subtopics in both the images in simple words so that i can answer any mcq questions that comes in exam from this topic...donot miss any concept...explain everything there is about these topics

Below is a **complete, exam-oriented, simple explanation** of **every subtopic shown in both images** from **"Interacting with the Oracle Server (PL/SQL)"**.
I'll explain **all concepts**, **rules**, **keywords**, **differences**, and **MCQ-traps**, in **very simple language**, so you can confidently answer **any MCQ** from this chapter.

CHAPTER 18 – Interacting with the Oracle Server (PL/SQL)

This chapter explains **how PL/SQL programs communicate with the Oracle database** using SQL statements.

---

1. Objectives

After studying this chapter, you should be able to:

- Use **SQL statements inside PL/SQL**
- Retrieve data using **SELECT INTO**
- Modify data using **INSERT, UPDATE, DELETE**
- Use **COMMIT and ROLLBACK**
- Understand **SQL cursors**

- Use **cursor attributes**
- Follow **naming conventions**

📌 **MCQ Tip**: Objectives questions are usually direct or conceptual.

---

2. SQL Statements in PL/SQL

What does this mean?

PL/SQL allows you to write **SQL statements inside a PL/SQL block**.

Types of SQL statements allowed:

1. **DML (Data Manipulation Language)**
   - `SELECT`
   - `INSERT`
   - `UPDATE`
   - `DELETE`
2. **TCL (Transaction Control Language)**
   - `COMMIT`
   - `ROLLBACK`
   - `SAVEPOINT`

❌ **DDL (CREATE, DROP, ALTER)** are **NOT allowed directly** in PL/SQL
(They must be used with `EXECUTE IMMEDIATE`)

📌 **MCQ Trap**:

> *DDL statements cannot be directly used inside PL/SQL.*

---

3. SELECT Statements in PL/SQL

In PL/SQL, **SELECT must always use INTO**.

Syntax:

```
sql

SELECT column_name
INTO variable_name
FROM table_name
WHERE condition;
```

Rules:

- SELECT must return **ONLY ONE ROW**

- SELECT must use **INTO**
- Variables must be declared before use

Example:

```
sql

SELECT salary
INTO v_salary
FROM employees
WHERE employee_id = 101;
```

❌ **Error if**:

- No row found → **`NO_DATA_FOUND`**
- More than one row → **`TOO_MANY_ROWS`**

📌 **MCQ Tip**:

SELECT without INTO is invalid in PL/SQL

---

4. Retrieving Data in PL/SQL

Ways to retrieve data:

1. **Single-row SELECT INTO**
2. **Cursors** (for multiple rows)

Example:

```
sql

DECLARE
  v_name employees.first_name%TYPE;
BEGIN
  SELECT first_name INTO v_name
  FROM employees
  WHERE employee_id = 100;
END;
```

📌 **Key Point**:

- PL/SQL variables store retrieved values
- **`%TYPE`** ensures datatype consistency

---

5. Manipulating Data Using PL/SQL

PL/SQL allows data manipulation using:

- INSERT

- UPDATE
- DELETE

These changes are **temporary** until committed.

📌 **MCQ Trap**:

> *DML changes are not permanent until COMMIT.*

---

6. Inserting Data

Syntax:

```sql
INSERT INTO table_name VALUES (...);
```

OR

```sql
INSERT INTO table_name (col1, col2) VALUES (...);
```

Example:

```sql
INSERT INTO employees (employee_id, first_name)
VALUES (110, 'Ravi');
```

📌 **Important**:

- Column count must match value count
- Data types must match

---

7. Updating Data

Syntax:

```sql
UPDATE table_name
SET column = value
WHERE condition;
```

Example:

```sql
UPDATE employees
SET salary = salary + 1000
WHERE employee_id = 101;
```

📌 **MCQ Trap**:

> *Without WHERE clause, all rows get updated.*

---

8. Deleting Data

Syntax:

```sql
DELETE FROM table_name WHERE condition;
```

Example:

```sql
DELETE FROM employees WHERE employee_id = 110;
```

📌 **Important**:

- DELETE without WHERE deletes **all rows**
- Table structure remains

---

9. Naming Conventions

Naming conventions make code readable and reduce errors.

Common conventions:

Type	Prefix
Variable	**v_**
Constant	**c_**
Cursor	**cur_**
Parameter	**p_**

Example:

```sql

v_salary NUMBER;
cur_emp SYS_REFCURSOR;
```

📌 **MCQ Tip**:

Naming conventions are **recommended**, not mandatory.

---

10. COMMIT and ROLLBACK Statements

COMMIT

- Saves changes permanently
- Ends the transaction

```sql

COMMIT;
```

ROLLBACK

- Undo changes
- Goes back to last COMMIT

```sql

ROLLBACK;
```

SAVEPOINT

- Marks a point to rollback partially

```sql

SAVEPOINT sp1;
ROLLBACK TO sp1;
```

📌 **MCQ Traps**:

- COMMIT makes changes permanent
- ROLLBACK cannot undo committed changes

---

11. SQL Cursor

What is a Cursor?

A cursor is a **pointer to the result set** of a query.

Types of cursors:

1. **Implicit Cursor**
   - Automatically created
   - Used for INSERT, UPDATE, DELETE, single-row SELECT
2. **Explicit Cursor**
   - Created by programmer
   - Used for multi-row SELECT

---

12. Implicit Cursor

Used automatically by Oracle.

Cursor name:

```
sql

SQL
```

Example:

```
sql

UPDATE employees SET salary = salary + 1000;
```

Oracle automatically creates an implicit cursor.

---

13. SQL Cursor Attributes

Cursor attributes give information about SQL execution.

Common Attributes:

Attribute	Meaning
SQL%FOUND	TRUE if rows affected
SQL%NOTFOUND	TRUE if no rows affected
SQL%ROWCOUNT	Number of rows affected
SQL%ISOPEN	Always FALSE for implicit cursor

Example:

```sql
IF SQL%FOUND THEN
  DBMS_OUTPUT.PUT_LINE('Rows updated');
END IF;
```

📌 **MCQ Trap**:

> **`SQL%ISOPEN`** is **always FALSE** for implicit cursor

---

14. Summary

You learned:

- How PL/SQL interacts with Oracle DB
- SELECT INTO rules
- DML operations
- Transaction control
- Cursor basics
- Cursor attributes
- Naming standards

---

15. Practice Overview

Practice includes:

- Writing PL/SQL blocks
- Using SELECT INTO
- Using INSERT/UPDATE/DELETE
- Handling COMMIT & ROLLBACK
- Using SQL cursor attributes

📌 **MCQ Tip**:
Practice questions usually test **rules, restrictions, and errors**.

MOST IMPORTANT MCQ QUICK REVISION

✔️ SELECT must use INTO
✔️ SELECT must return only one row
✔️ DDL not allowed directly in PL/SQL
✔️ COMMIT saves permanently
✔️ ROLLBACK undoes uncommitted work
✔️ Implicit cursor is **`SQL`**

✔️ SQL%ROWCOUNT gives affected rows
✔️ SQL%ISOPEN always FALSE

If you want, I can also:

- ✅ Give **MCQs with answers**
- ✅ Create **1-page exam notes**
- ✅ Explain **errors & exceptions**
- ✅ Give **PL/SQL programs with explanation**

Just tell me 👍

> explain all cursor attributes

Below is a **complete, exam-ready explanation of *ALL cursor attributes*** in **PL/SQL**, explained in **very simple language**, with **rules, examples, and MCQ traps**.
After this, you should be able to answer **any MCQ** related to cursor attributes confidently.

CURSOR ATTRIBUTES IN PL/SQL

Cursor attributes give **information about the execution of a SQL statement or cursor**.

They tell us things like:

- Did any row get affected?
- How many rows were affected?
- Is the cursor open or closed?

TYPES OF CURSOR ATTRIBUTES

There are **two categories**:

1. **Implicit Cursor Attributes**
2. **Explicit Cursor Attributes**

---

1. IMPLICIT CURSOR ATTRIBUTES

- Implicit cursors are **automatically created by Oracle**
- Cursor name is **`SQL`**

Used with:

- `INSERT`
- `UPDATE`
- `DELETE`
- `SELECT INTO` (single row)

Implicit Cursor Attributes List

Attribute	Meaning
`SQL%FOUND`	TRUE if at least one row affected
`SQL%NOTFOUND`	TRUE if no row affected
`SQL%ROWCOUNT`	Number of rows affected
`SQL%ISOPEN`	Always FALSE

---

1. SQL%FOUND

What it does:

- Returns **TRUE** if **one or more rows** are affected
- Returns **FALSE** if **no rows** are affected

Example:

```
sql


UPDATE employees SET salary = salary + 1000
WHERE department_id = 10;

IF SQL%FOUND THEN
  DBMS_OUTPUT.PUT_LINE('Rows updated');
END IF;
```

MCQ Point:

✔️ TRUE → at least 1 row affected
❌ FALSE → 0 rows affected

---

2. SQL%NOTFOUND

What it does:

- Returns **TRUE** if **no rows** are affected
- Opposite of `%FOUND`

Example:

```
sql

DELETE FROM employees WHERE employee_id = 999;

IF SQL%NOTFOUND THEN
  DBMS_OUTPUT.PUT_LINE('No rows deleted');
END IF;
```

MCQ Trap:

`SQL%NOTFOUND = NOT SQL%FOUND`

---

3. SQL%ROWCOUNT

What it does:

- Returns the **number of rows affected**
- Returns a **number**

Example:

```
sql

UPDATE employees SET salary = salary + 500;

DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' rows updated');
```

Important Rules:

- After SELECT INTO → returns **1**
- After no row affected → returns **0**

---

4. SQL%ISOPEN

What it does:

- Checks whether cursor is open

Implicit Cursor Rule:

❗ **ALWAYS FALSE**

Why?

- Oracle opens and closes implicit cursors automatically

MCQ Trap:

`SQL%ISOPEN` is **always FALSE** for implicit cursors

---

2. EXPLICIT CURSOR ATTRIBUTES

- Explicit cursors are **defined by the programmer**
- Used for **multiple-row SELECT queries**

Explicit Cursor Attributes List

Attribute	Meaning
`cursor_name%FOUND`	TRUE if last fetch returned a row
`cursor_name%NOTFOUND`	TRUE if last fetch returned no row
`cursor_name%ROWCOUNT`	Number of rows fetched
`cursor_name%ISOPEN`	TRUE if cursor is open

Explicit Cursor Life Cycle

1. **DECLARE** cursor
2. **OPEN** cursor
3. **FETCH** data
4. **CLOSE** cursor

---

1. cursor_name%FOUND

What it does:

- TRUE → last FETCH retrieved a row
- FALSE → no row fetched

Example:

```
sql

FETCH cur_emp INTO v_id;

IF cur_emp%FOUND THEN
  DBMS_OUTPUT.PUT_LINE('Row fetched');
END IF;
```

---

2. cursor_name%NOTFOUND

What it does:

- TRUE → last FETCH did NOT retrieve a row
- Used to exit loops

Example:

```
sql

LOOP
  FETCH cur_emp INTO v_id;
  EXIT WHEN cur_emp%NOTFOUND;
END LOOP;
```

MCQ Tip:

✔️ **%NOTFOUND** used to end cursor loop

---

3. cursor_name%ROWCOUNT

What it does:

- Returns number of rows fetched so far

Example:

```
sql

DBMS_OUTPUT.PUT_LINE(cur_emp%ROWCOUNT);
```

Important:

- Starts from **0**
- Increases after every FETCH

---

4. cursor_name%ISOPEN

What it does:

- TRUE → cursor is open
- FALSE → cursor is closed

Example:

```
sql

IF cur_emp%ISOPEN THEN
  DBMS_OUTPUT.PUT_LINE('Cursor is open');
```

```
END IF;
```

IMPLICIT vs EXPLICIT CURSOR ATTRIBUTES (MCQ FAVORITE)

Feature	Implicit	Explicit
Cursor name	SQL	User-defined
%FOUND	Yes	Yes
%NOTFOUND	Yes	Yes
%ROWCOUNT	Yes	Yes
%ISOPEN	Always FALSE	TRUE / FALSE

COMMON MCQ TRAPS

❌ **`SQL%ISOPEN = TRUE`** → WRONG
✔️ Always FALSE

❌ **`%ROWCOUNT`** gives total rows in table → WRONG
✔️ Only affected rows

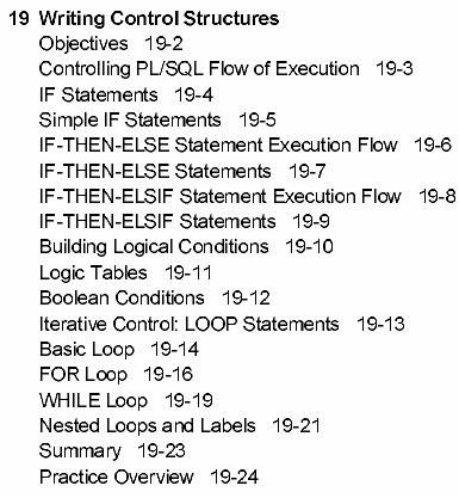
❌ **`%FOUND`** checks table data → WRONG
✔️ Checks last SQL execution

1-LINE EXAM MEMORY TRICK

- **FOUND** → Something happened
- **NOTFOUND** → Nothing happened
- **ROWCOUNT** → How many
- **ISOPEN** → Cursor status

If you want next, I can:
✅ Give **20 MCQs with answers**
✅ Explain **explicit cursor programs line-by-line**
✅ Explain **cursor FOR loop (exam favorite)**

Just tell me 👍



19 Writing Control Structures
Objectives 19-2
Controlling PL/SQL Flow of Execution 19-3
IF Statements 19-4
Simple IF Statements 19-5
IF-THEN-ELSE Statement Execution Flow 19-6
IF-THEN-ELSE Statements 19-7
IF-THEN-ELSIF Statement Execution Flow 19-8
IF-THEN-ELSIF Statements 19-9
Building Logical Conditions 19-10
Logic Tables 19-11
Boolean Conditions 19-12
Iterative Control: LOOP Statements 19-13
Basic Loop 19-14
FOR Loop 19-16
WHILE Loop 19-19
Nested Loops and Labels 19-21
Summary 19-23
Practice Overview 19-24

go through the contents and explain everything about subtopics in both the images in simple words so that i can answer any mcq questions that comes in exam from this topic...donot miss any concept...explain everything there is about these topics

Perfect 👍
This is **Chapter 19 – Writing Control Structures (PL/SQL)**.
Below is a **complete, exam-oriented, simple explanation of EVERY subtopic shown in the image**.
I'll cover **definitions, syntax, rules, execution flow, differences, common errors, and MCQ traps**.
After this, you should be able to answer **any MCQ** from this chapter confidently.

CHAPTER 19 – WRITING CONTROL STRUCTURES (PL/SQL)

Control structures decide **which statements run, when they run, and how many times they run**.

---

1. Objectives

After this chapter, you should be able to:

- Control the **flow of execution** in PL/SQL
- Use **IF conditions**
- Build **logical and Boolean expressions**
- Use **loops** (LOOP, WHILE, FOR)
- Use **nested loops and labels**

📌 *MCQ tip:* Objectives are direct theory questions.

---

2. Controlling PL/SQL Flow of Execution

By default, PL/SQL executes statements **top to bottom**.

Control structures allow:

- **Decision making** → IF statements
- **Repetition** → Loops

---

3. IF Statements

IF statements execute code **only when a condition is TRUE**.

Basic rule:

- Condition must return **TRUE or FALSE**
- NULL is treated as **FALSE**

---

4. Simple IF Statement

Syntax:

```sql
sql

IF condition THEN
   statement;
END IF;
```

Example:

```sql
sql

IF salary > 50000 THEN
   bonus := 1000;
END IF;
```

📌 **Key Rules (MCQ favorite)**:

- **`END IF;`** is **mandatory**
- Condition must be **Boolean**
- No ELSE part

---

5. IF–THEN–ELSE Statement Execution Flow

Executes:

- IF block → if condition TRUE
- ELSE block → if condition FALSE

Flow:

```sql
Check condition
→ TRUE → IF block
→ FALSE → ELSE block
```

---

6. IF–THEN–ELSE Statement

Syntax:

```sql
IF condition THEN
   statement1;
ELSE
   statement2;
END IF;
```

Example:

```sql
IF marks >= 40 THEN
   result := 'PASS';
ELSE
   result := 'FAIL';
END IF;
```

📌 **MCQ Trap**:

- ELSE executes when condition is **FALSE or NULL**

---

7. IF–THEN–ELSIF Statement Execution Flow

Used to check **multiple conditions in sequence**.

Execution stops at the **first TRUE condition**.

---

8. IF–THEN–ELSIF Statement

Syntax:

```sql
IF condition1 THEN
   statement1;
ELSIF condition2 THEN
   statement2;
ELSIF condition3 THEN
   statement3;
ELSE
```

```
   statement4;
END IF;
```

Example:

```
sql

IF marks >= 90 THEN
   grade := 'A';
ELSIF marks >= 75 THEN
   grade := 'B';
ELSIF marks >= 60 THEN
   grade := 'C';
ELSE
   grade := 'F';
END IF;
```

📌 **Rules**:

- ELSE is **optional**
- ELSIF can be **multiple**
- Only **one block executes**

---

9. Building Logical Conditions

Logical conditions combine comparisons using **logical operators**.

Comparison Operators:

Operator	Meaning
=	equal
!= or <>	not equal
>	greater than
<	less than
>=	greater or equal
<=	less or equal

---

10. Logic Tables (Logical Operators)

AND Operator

A	B	A AND B
T	T	T
T	F	F
F	T	F
F	F	F

OR Operator

A	B	A OR B
T	T	T
T	F	T
F	T	T
F	F	F

NOT Operator

A	NOT A
T	F
F	T

📌 **MCQ Trap**:

- AND needs **both TRUE**
- OR needs **any TRUE**

---

11. Boolean Conditions

Boolean datatype values:

- TRUE
- FALSE
- NULL

Important rule:

> NULL in IF condition is treated as **FALSE**

Example:

```
sql

IF flag THEN
   ...
END IF;
```

If `flag = NULL`, IF block **will NOT execute**.

---

12. Iterative Control: LOOP Statements

Loops repeat code **multiple times**.

Types of loops:

1. **Basic LOOP**
2. **WHILE LOOP**
3. **FOR LOOP**

---

13. Basic LOOP

Syntax:

```sql
LOOP
   statements;
   EXIT WHEN condition;
END LOOP;
```

Example:

```sql
LOOP
   i := i + 1;
   EXIT WHEN i = 5;
END LOOP;
```

📌 **Rules**:

- EXIT condition is **mandatory**
- Otherwise → **infinite loop**

---

14. FOR LOOP

Syntax:

```sql
FOR i IN lower..upper LOOP
   statements;
END LOOP;
```

Example:

```sql
FOR i IN 1..5 LOOP
   DBMS_OUTPUT.PUT_LINE(i);
```

```
END LOOP;
```

📌 **Important FOR Loop Rules (Very Important for MCQ)**:
✔️ Loop variable is **automatically declared**
✔️ Loop variable is **read-only**
✔️ Loop runs **fixed number of times**

---

15. WHILE LOOP

Syntax:

```sql
WHILE condition LOOP
   statements;
END LOOP;
```

Example:

```sql
WHILE i <= 5 LOOP
   i := i + 1;
END LOOP;
```

📌 **Rules**:

- Condition checked **before execution**
- May execute **zero times**

---

16. Nested Loops and Labels

Nested Loop:

Loop inside another loop.

```sql
FOR i IN 1..3 LOOP
   FOR j IN 1..2 LOOP
      DBMS_OUTPUT.PUT_LINE(i || ',' || j);
   END LOOP;
END LOOP;
```

---

17. Labels

Labels help identify loops, especially in nested loops.

Syntax:

```sql
<<label_name>>
LOOP
   EXIT label_name WHEN condition;
END LOOP label_name;
```

Example:

```sql
<<outer>>
FOR i IN 1..3 LOOP
   FOR j IN 1..3 LOOP
      EXIT outer WHEN i = 2;
   END LOOP;
END LOOP outer;
```

📌 **MCQ Trap**:

- Labels allow **exiting outer loop from inner loop**

---

18. Summary

You learned:

- IF / IF-ELSE / IF-ELSIF
- Boolean and logical expressions
- LOOP, WHILE, FOR
- Nested loops and labels

---

19. Practice Overview

Practice includes:

- Writing IF conditions
- Using logical operators
- Writing loop programs
- Finding output of loops (MCQ favorite)

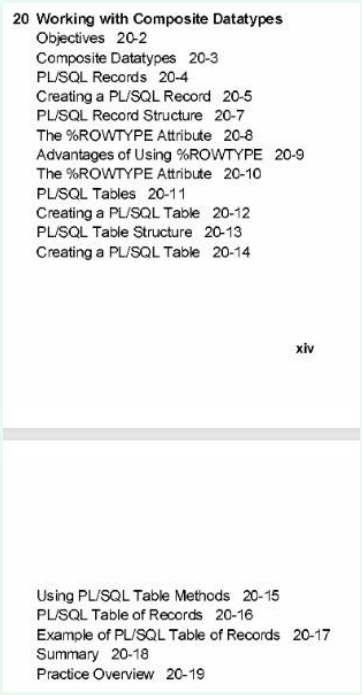
VERY IMPORTANT MCQ QUICK REVISION

✔️ IF condition must be Boolean
✔️ NULL in IF = FALSE
✔️ ELSE is optional
✔️ FOR loop variable is read-only
✔️ WHILE loop may run zero times

✔️ Basic LOOP needs EXIT
✔️ Labels can exit outer loops

If you want next, I can:
✅ Give **30 MCQs with answers**
✅ Explain **loop output prediction questions**
✅ Create **1-page exam cheat sheet**

Just tell me 👍



go through the contents and explain everything about subtopics in both the images in simple words so that i can answer any mcq questions that comes in exam from this topic...donot miss any concept...explain everything there is about these topics

Excellent 👍
This is **Chapter 20 – Working with Composite Datatypes (PL/SQL)**.
Below is a **complete, exam-oriented, simple explanation of EVERY subtopic shown in BOTH images**.
I'll cover **definitions, syntax, rules, differences, advantages, examples, and MCQ traps**.
After this, you should be able to answer **any MCQ** from this chapter confidently.

CHAPTER 20 – WORKING WITH COMPOSITE DATATYPES

Composite datatypes are datatypes that can **store more than one value**.

---

1. Objectives

After this chapter, you should be able to:

- Understand composite datatypes
- Create and use **PL/SQL records**
- Use **%ROWTYPE**
- Create and use **PL/SQL tables (collections)**
- Use table methods
- Work with **tables of records**

📌 *MCQ tip:* Objectives are direct theory questions.

---

2. Composite Datatypes

What are Composite Datatypes?

Datatypes that contain **multiple fields or values**.

Types in PL/SQL:

1. **RECORD**
2. **PL/SQL Table (Collection)**

📌 **MCQ Trap**:

> Composite datatype can store multiple values of **same or different datatypes**

---

3. PL/SQL Records

What is a Record?

A record is a **group of related fields** (like a row in a table).

Example:

- employee_id
- name
- salary

All stored in **one variable**.

---

4. Creating a PL/SQL Record

Syntax:

```sql

TYPE record_type IS RECORD (
   field1 datatype,
   field2 datatype
);

record_variable record_type;
```

Example:

```sql

TYPE emp_rec IS RECORD (
   emp_id NUMBER,
   emp_name VARCHAR2(50),
   salary NUMBER
);

v_emp emp_rec;
```

📌 **MCQ Tip**:

- TYPE must be declared **before** record variable

---

5. PL/SQL Record Structure

Access fields using **dot notation**.

```sql

v_emp.emp_id := 101;
v_emp.emp_name := 'Ravi';
v_emp.salary := 50000;
```

📌 **Key Rule**:

> Record fields are accessed using **`record_name.field_name`**

---

6. The %ROWTYPE Attribute

What is %ROWTYPE?

Creates a record with **same structure as a table row**.

Syntax:

```sql

variable_name table_name%ROWTYPE;
```

Example:

```sql
v_emp employees%ROWTYPE;
```

📌 **Important**:

- Automatically matches table columns
- Includes **all columns**

---

7. Advantages of Using %ROWTYPE

Advantage	Explanation
No datatype mismatch	Matches table structure
Less coding	No need to define each field
Auto update	Changes if table changes

📌 **MCQ Trap**:

> **%ROWTYPE** adapts automatically if table structure changes

---

8. The %ROWTYPE Attribute (Usage)

Fetching data:

```sql
SELECT * INTO v_emp FROM employees WHERE employee_id = 101;
```

📌 **Rule**:

- SELECT must return **exactly one row**

---

9. PL/SQL Tables (Collections)

What is a PL/SQL Table?

A **collection of elements** stored in memory.

Features:

- Indexed by integers
- Can grow dynamically
- Similar to arrays

---

10. Creating a PL/SQL Table

Syntax:

```
sql

TYPE table_type IS TABLE OF datatype
INDEX BY PLS_INTEGER;

table_variable table_type;
```

Example:

```
sql

TYPE salary_tab IS TABLE OF NUMBER
INDEX BY PLS_INTEGER;

v_salary salary_tab;
```

📌 **MCQ Tip**:

- Index starts from **any integer**
- Not stored in database

---

11. PL/SQL Table Structure

Access elements using index:

```
sql

v_salary(1) := 10000;
v_salary(2) := 20000;
```

---

12. Creating a PL/SQL Table (Again)

PL/SQL tables are:

- **Memory based**
- **Temporary**
- Exist only during session

📌 **MCQ Trap**:

> PL/SQL tables are not database tables

---

13. Using PL/SQL Table Methods

Common Methods:

Method	Meaning
COUNT	Number of elements
FIRST	First index
LAST	Last index
NEXT(i)	Next index
PRIOR(i)	Previous index
EXISTS(i)	Checks existence

Example:

```sql
sql

IF v_salary.EXISTS(1) THEN
   DBMS_OUTPUT.PUT_LINE(v_salary(1));
END IF;
```

📌 **MCQ Favorite**:

- EXISTS prevents **NO_DATA_FOUND**

---

14. PL/SQL Table of Records

Meaning:

A collection where **each element is a record**.

Syntax:

```sql
sql

TYPE emp_rec IS RECORD (
   emp_id NUMBER,
   name VARCHAR2(50)
);

TYPE emp_tab IS TABLE OF emp_rec
INDEX BY PLS_INTEGER;

v_emp emp_tab;
```

---

15. Example of PL/SQL Table of Records

```sql
sql

v_emp(1).emp_id := 101;
v_emp(1).name := 'Ravi';
```

📌 **MCQ Trap**:

Access = `table(index).field`

---

16. Summary

You learned:

- Composite datatypes
- Records
- %ROWTYPE
- PL/SQL tables
- Table methods
- Tables of records

---

17. Practice Overview

Practice includes:

- Creating records
- Using %ROWTYPE
- Using collections
- Using methods
- Output-based MCQs

VERY IMPORTANT MCQ QUICK REVISION

✔️ Composite datatype stores multiple values
✔️ Record = group of fields
✔️ `%ROWTYPE` matches table row
✔️ PL/SQL tables are memory-based
✔️ Index by PLS_INTEGER
✔️ EXISTS checks index existence
✔️ Table of records uses dot + index

If you want next, I can:
✅ Create **MCQs with solutions**
✅ Explain **difference between RECORD & %ROWTYPE**
✅ Provide **1-page exam notes**

Just tell me 👍

looping with records

Below is a **complete, exam-ready explanation of *Looping with Records in PL/SQL***, written in **very simple words**, covering **all concepts, syntax, rules, examples, and MCQ traps**.
After this, you should be able to answer **any MCQ** from this topic confidently.

LOOPING WITH RECORDS IN PL/SQL

Looping with records means **repeating operations while working with records** (single record, **`%ROWTYPE`**, or record collections).

---

1. What is a Record (Quick Recall)

A **record** is a composite datatype that stores **multiple fields together**.

Example:

```sql
sql

TYPE emp_rec IS RECORD (
   emp_id NUMBER,
   name   VARCHAR2(50),
   salary NUMBER
);
```

---

2. Why Loop with Records?

We loop with records when:

- We fetch **multiple rows** from a table
- We process **each row one by one**
- We work with **collections of records**

📌 **MCQ Tip**:

> Records + loops = multi-row processing

WAYS TO LOOP WITH RECORDS

There are **3 exam-important ways**:

1. **Cursor FOR loop with records** ⭐ (MOST IMPORTANT)
2. **Explicit cursor + LOOP + FETCH**
3. **Looping through PL/SQL table (collection) of records**

1. Cursor FOR Loop with Records (⭐ EXAM FAVORITE)

Key Idea

- Oracle **automatically creates a record**
- Record structure matches the SELECT list
- No need to OPEN, FETCH, or CLOSE cursor

Syntax

```
sql

FOR record_name IN (SELECT columns FROM table) LOOP
   statements;
END LOOP;
```

Example

```
sql

FOR emp_rec IN (SELECT employee_id, salary FROM employees) LOOP
   DBMS_OUTPUT.PUT_LINE(emp_rec.employee_id || ' ' || emp_rec.salary);
END LOOP;
```

Important Rules (MCQ gold):

✔️ Record is **implicitly declared**
✔️ Record fields match SELECT columns
✔️ Record is **read-only**
✔️ Cursor is opened & closed automatically

MCQ Trap ❌

> Can we assign values to emp_rec.salary?
> ❌ NO (read-only)

---

2. Explicit Cursor + LOOP + FETCH (Manual Looping)

Used when:

- More control is needed
- Logic is complex

Syntax

```sql
DECLARE
   CURSOR c1 IS SELECT * FROM employees;
   v_emp employees%ROWTYPE;
BEGIN
   OPEN c1;
   LOOP
      FETCH c1 INTO v_emp;
      EXIT WHEN c1%NOTFOUND;

      DBMS_OUTPUT.PUT_LINE(v_emp.employee_id);
   END LOOP;
   CLOSE c1;
END;
```

Rules:

✔️ FETCH loads **one row into record**
✔️ **%NOTFOUND** ends loop
✔️ OPEN & CLOSE required

MCQ Trap:

> Forgetting EXIT WHEN → **infinite loop**

---

3. Looping with %ROWTYPE Records

Idea

- **%ROWTYPE** record holds **entire row**
- Used with cursor or SELECT INTO

Example

```sql
DECLARE
   v_emp employees%ROWTYPE;
BEGIN
   FOR v_emp IN (SELECT * FROM employees) LOOP
      DBMS_OUTPUT.PUT_LINE(v_emp.first_name);
   END LOOP;
END;
```

📌 **MCQ Tip**:

> **%ROWTYPE** record automatically matches table structure

---

4. Looping Through PL/SQL Table (Collection) of Records

When used?

- Data stored in memory
- Batch processing

Structure

```sql
TYPE emp_rec IS RECORD (
   id NUMBER,
   name VARCHAR2(50)
);

TYPE emp_tab IS TABLE OF emp_rec INDEX BY PLS_INTEGER;
v_emp emp_tab;
```

Looping Example

```sql
FOR i IN v_emp.FIRST .. v_emp.LAST LOOP
   DBMS_OUTPUT.PUT_LINE(v_emp(i).id);
END LOOP;
```

Safe Loop (Best Practice)

```sql
i := v_emp.FIRST;
WHILE i IS NOT NULL LOOP
   DBMS_OUTPUT.PUT_LINE(v_emp(i).name);
   i := v_emp.NEXT(i);
END LOOP;
```

📌 **MCQ Trap**:

> Collection indexes may be **non-continuous**

COMPARISON (VERY IMPORTANT FOR MCQs)

Method	Best Use	Auto Cursor
Cursor FOR loop	Simple row processing	Yes
Explicit cursor	Complex logic	No
Collection loop	In-memory data	No

COMMON MCQ TRAPS

❌ Cursor FOR loop needs OPEN → WRONG
✔️ Automatic

❌ Record fields can be updated in FOR loop → WRONG

❌ %ROWTYPE stores single column → WRONG

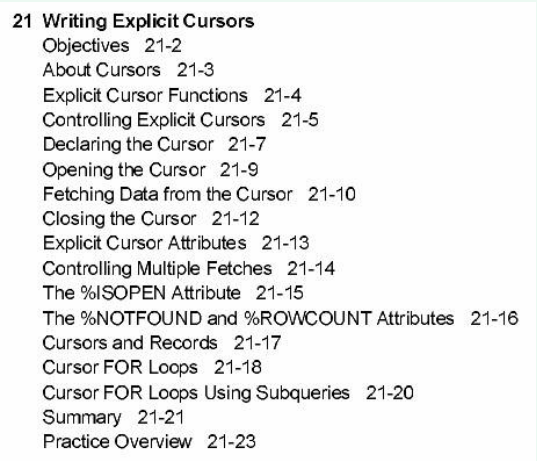
❌ Collection index always starts at 1 → WRONG

1-LINE EXAM MEMORY TRICK

- **Cursor FOR loop** → easiest & safest
- **Explicit cursor** → full control
- **Collection loop** → memory data

If you want next, I can:
✅ Give **MCQs with answers**
✅ Explain **output-based questions**
✅ Create **diagram-based explanation**
✅ Compare **records vs collections vs cursors**

Just tell me 👍



21 Writing Explicit Cursors
Objectives 21-2
About Cursors 21-3
Explicit Cursor Functions 21-4
Controlling Explicit Cursors 21-5
Declaring the Cursor 21-7
Opening the Cursor 21-9
Fetching Data from the Cursor 21-10
Closing the Cursor 21-12
Explicit Cursor Attributes 21-13
Controlling Multiple Fetches 21-14
The %ISOPEN Attribute 21-15
The %NOTFOUND and %ROWCOUNT Attributes 21-16
Cursors and Records 21-17
Cursor FOR Loops 21-18
Cursor FOR Loops Using Subqueries 21-20
Summary 21-21
Practice Overview 21-23

explain all these topics in the same way

Perfect 👍
This is **Chapter 21 – Writing Explicit Cursors (PL/SQL)**.
Below is a **complete, exam-oriented, simple explanation of EVERY subtopic listed**, explained in the **same style as previous chapters** — clear language, step-by-step flow, rules, examples, and **MCQ traps**.
After this, you'll be able to answer **any MCQ** from this chapter confidently.

CHAPTER 21 – WRITING EXPLICIT CURSORS

Explicit cursors are used when a **SELECT statement returns more than one row** and you want **full control** over row processing.

---

1. Objectives

After this chapter, you should be able to:

- Understand what cursors are
- Write explicit cursors
- Control cursor execution
- Use cursor attributes
- Use cursor FOR loops
- Work with cursors and records

---

2. About Cursors

What is a Cursor?

A **cursor** is a pointer to the **result set** of a SQL query.

Why cursors are needed?

- `SELECT INTO` → **only one row**
- Multiple rows → **cursor required**

---

3. Explicit Cursor Functions (What Explicit Cursor Does)

Explicit cursors allow you to:

- Process **multiple rows**
- Fetch rows **one at a time**
- Control execution manually

📌 **MCQ Tip**:

> Explicit cursors are programmer-controlled.

---

4. Controlling Explicit Cursors

Explicit cursor execution has **4 steps**:

1. **DECLARE** cursor
2. **OPEN** cursor

3. **FETCH** data
4. **CLOSE** cursor

📌 **MCQ Favorite**:

> Correct order = Declare → Open → Fetch → Close

---

5. Declaring the Cursor

Syntax:

```sql
CURSOR cursor_name IS
   SELECT statement;
```

Example:

```sql
CURSOR c_emp IS
   SELECT employee_id, salary FROM employees;
```

📌 **Rules**:

- Cursor declared in **declarative section**
- SELECT can return **multiple rows**

---

6. Opening the Cursor

Syntax:

```sql
OPEN cursor_name;
```

What happens when cursor opens?

- SQL is executed
- Result set is created
- Cursor points **before first row**

📌 **MCQ Trap**:

> No data is fetched during OPEN

---

7. Fetching Data from the Cursor

Syntax:

```
sql

FETCH cursor_name INTO variable(s);
```

Example:

```
sql

FETCH c_emp INTO v_id, v_salary;
```

📌 **Rules**:

- FETCH retrieves **one row at a time**
- INTO variables must match SELECT list

---

8. Closing the Cursor

Syntax:

```
sql

CLOSE cursor_name;
```

Why close cursor?

- Frees memory
- Good programming practice

📌 **MCQ Tip**:

> Cursor must be closed after use

---

9. Explicit Cursor Attributes

Cursor attributes give **status information**.

Attribute	Meaning
%FOUND	Last fetch returned row
%NOTFOUND	Last fetch returned no row
%ROWCOUNT	Rows fetched so far
%ISOPEN	Cursor open or not

10. Controlling Multiple Fetches

Typical Loop:

```
sql

LOOP
   FETCH c_emp INTO v_id, v_salary;
   EXIT WHEN c_emp%NOTFOUND;
   DBMS_OUTPUT.PUT_LINE(v_id);
END LOOP;
```

📌 **MCQ Trap**:

EXIT WHEN must be **after FETCH**

---

11. The %ISOPEN Attribute

Meaning:

- TRUE → cursor open
- FALSE → cursor closed

Example:

```
sql

IF c_emp%ISOPEN THEN
   DBMS_OUTPUT.PUT_LINE('Cursor is open');
END IF;
```

📌 **MCQ Difference**:

- Explicit cursor → TRUE/FALSE
- Implicit cursor → always FALSE

---

12. The %NOTFOUND and %ROWCOUNT Attributes

%NOTFOUND

- TRUE when no row fetched

%ROWCOUNT

- Number of rows fetched so far

📌 **MCQ Trap**:

%ROWCOUNT increases **after each FETCH**

13. Cursors and Records

Using Records with Cursor

Instead of multiple variables, use **record**.

```sql
CURSOR c_emp IS SELECT * FROM employees;
v_emp employees%ROWTYPE;
```

```sql
FETCH c_emp INTO v_emp;
```

📌 **MCQ Tip**:

> Record structure must match cursor SELECT list

---

14. Cursor FOR Loops (⭐ MOST IMPORTANT)

Why Cursor FOR Loop?

- Simplest way to process multiple rows
- No OPEN / FETCH / CLOSE

Syntax:

```sql
FOR record_name IN cursor_name LOOP
   statements;
END LOOP;
```

OR

```sql
FOR record_name IN (SELECT ...) LOOP
   statements;
END LOOP;
```

Example:

```sql
FOR emp_rec IN c_emp LOOP
   DBMS_OUTPUT.PUT_LINE(emp_rec.employee_id);
```

```
END LOOP;
```

📌 **MCQ Gold Rules**:
✔️ Cursor automatically opened
✔️ Record automatically created
✔️ Record is read-only
✔️ Cursor automatically closed

---

15. Cursor FOR Loops Using Subqueries

Example:

```
sql

FOR emp_rec IN (SELECT employee_id FROM employees WHERE salary > 50000) LOOP
   DBMS_OUTPUT.PUT_LINE(emp_rec.employee_id);
END LOOP;
```

📌 **MCQ Tip**:

> Cursor name not required when using subquery

---

16. Summary

You learned:

- Explicit cursor basics
- Cursor life cycle
- Cursor attributes
- Records with cursors
- Cursor FOR loops

---

17. Practice Overview

Practice includes:

- Writing explicit cursor programs
- Finding output
- Error identification
- MCQs on attributes & flow

VERY IMPORTANT MCQ QUICK REVISION

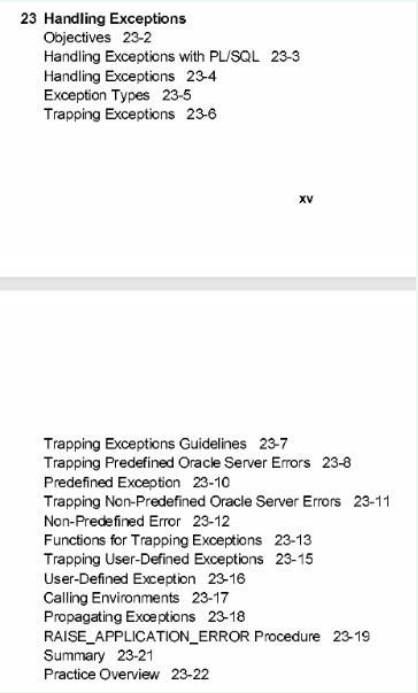
✔️ Explicit cursor handles multi-row SELECT
✔️ OPEN does not fetch data
✔️ FETCH retrieves one row
✔️ EXIT WHEN after FETCH
✔️ Cursor FOR loop auto-manages cursor
✔️ %ROWCOUNT increases per FETCH

1-LINE EXAM MEMORY TRICK

> **Explicit cursor = manual control**
> **Cursor FOR loop = automatic & safest**

If you want next, I can:
✅ Create **MCQs with solutions**
✅ Explain **cursor errors**
✅ Compare **explicit vs implicit cursors**
✅ Give **output-based practice**

Just tell me 👍


23 Handling Exceptions
Objectives 23-2
Handling Exceptions with PL/SQL 23-3
Handling Exceptions 23-4
Exception Types 23-5
Trapping Exceptions 23-6
xv
Trapping Exceptions Guidelines 23-7
Trapping Predefined Oracle Server Errors 23-8
Predefined Exception 23-10
Trapping Non-Predefined Oracle Server Errors 23-11
Non-Predefined Error 23-12
Functions for Trapping Exceptions 23-13
Trapping User-Defined Exceptions 23-15
User-Defined Exception 23-16
Calling Environments 23-17
Propagating Exceptions 23-18
RAISE_APPLICATION_ERROR Procedure 23-19
Summary 23-21
Practice Overview 23-22

along with these sobtopics explain all the errors that can cause in plsql

Perfect 👍
This is **Chapter 23 – Handling Exceptions (PL/SQL)**.
Below is a **FULL, exam-oriented explanation of ALL subtopics shown**, PLUS a **complete list of PL/SQL errors/exceptions that can occur**, explained in **simple words**, with **rules, examples, and MCQ traps**.
After this, you should be able to answer **any MCQ or theory question** from this chapter confidently.

CHAPTER 23 – HANDLING EXCEPTIONS (PL/SQL)

An **exception** is an **error that occurs during execution** of a PL/SQL program.

1. Objectives

After this chapter, you should be able to:

- Understand exceptions
- Handle runtime errors
- Use predefined and user-defined exceptions
- Propagate exceptions
- Use **`RAISE_APPLICATION_ERROR`**

📌 *MCQ Tip:* Objectives questions are direct.

2. Handling Exceptions with PL/SQL

PL/SQL provides a **structured way to catch and handle errors** so the program does not terminate abruptly.

Structure of PL/SQL Block:

```
sql


BEGIN
   -- executable statements
EXCEPTION
   -- exception handlers
END;
```

📌 **Key Rule (MCQ favorite)**:

> Exception section is **optional**, but if present it must be **after BEGIN section**

3. Handling Exceptions

When an error occurs:

1. Normal execution stops
2. Control transfers to **EXCEPTION section**
3. Matching exception handler is executed

4. Exception Types

PL/SQL exceptions are of **3 types**:

Type	Description
Predefined Exceptions	Built-in Oracle errors
Non-Predefined Exceptions	Oracle errors without names
User-Defined Exceptions	Defined by programmer

📌 **MCQ Trap**:

> All Oracle errors are not predefined

---

5. Trapping Exceptions

Trapping means **catching an error and handling it gracefully**.

Syntax:

```
sql

EXCEPTION
   WHEN exception_name THEN
      statements;
```

---

6. Trapping Exceptions – Guidelines

✔️ Specific exceptions first
✔️ **`WHEN OTHERS`** last
✔️ Do not hide errors unnecessarily
✔️ Display meaningful messages

📌 **MCQ Trap**:

> **`WHEN OTHERS`** must always be the **last handler**

---

7. Trapping Predefined Oracle Server Errors

Predefined exceptions are **automatically raised by Oracle**.

Common Predefined Exceptions (VERY IMPORTANT)

Exception	Cause
`NO_DATA_FOUND`	SELECT INTO returns no row
`TOO_MANY_ROWS`	SELECT INTO returns multiple rows
`ZERO_DIVIDE`	Division by zero
`INVALID_NUMBER`	Invalid number conversion
`VALUE_ERROR`	Size/type mismatch

Exception	Cause
DUP_VAL_ON_INDEX	Duplicate primary key
LOGIN_DENIED	Invalid login
CURSOR_ALREADY_OPEN	Cursor opened twice
INVALID_CURSOR	Illegal cursor operation

Example:

```sql

EXCEPTION
   WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('No data found');
```

📌 **MCQ Favorite**:

> **NO_DATA_FOUND** occurs only with **SELECT INTO**

---

8. Predefined Exception (Concept)

- Already defined in Oracle
- No need to declare
- Raised automatically

---

9. Trapping Non-Predefined Oracle Server Errors

Some Oracle errors **do not have predefined names**.

To trap them:

1. Declare exception
2. Use **PRAGMA EXCEPTION_INIT**

---

10. Non-Predefined Error

Example Error:

```sql

ORA-01403
```

Handling:

```sql
DECLARE
   e_my_error EXCEPTION;
   PRAGMA EXCEPTION_INIT(e_my_error, -1403);
BEGIN
   ...
EXCEPTION
   WHEN e_my_error THEN
      DBMS_OUTPUT.PUT_LINE('Handled');
END;
```

📌 **MCQ Trap**:

Error number must be **negative**

---

11. Functions for Trapping Exceptions

Oracle provides **two functions**:

1. SQLCODE

- Returns error number

2. SQLERRM

- Returns error message

Example:

```sql
WHEN OTHERS THEN
   DBMS_OUTPUT.PUT_LINE(SQLCODE || ' ' || SQLERRM);
```

📌 **MCQ Tip**:

SQLERRM returns text message

---

12. Trapping User-Defined Exceptions

User-defined exceptions are created by programmer for **business rules**.

Steps:

1. Declare exception
2. Raise exception
3. Handle exception

---

13. User-Defined Exception

Example:

```
sql

DECLARE
   e_low_salary EXCEPTION;
BEGIN
   IF salary < 10000 THEN
      RAISE e_low_salary;
   END IF;
EXCEPTION
   WHEN e_low_salary THEN
      DBMS_OUTPUT.PUT_LINE('Salary too low');
END;
```

📌 **MCQ Trap**:

> User-defined exceptions must be raised manually

---

14. Calling Environments

When PL/SQL blocks call other blocks:

- Errors propagate **upward**
- If not handled, calling environment handles it

📌 **MCQ Tip**:

> Unhandled exception moves to calling block

---

15. Propagating Exceptions

Exception propagation means:

- Exception not handled in current block
- Passed to outer block

---

16. RAISE_APPLICATION_ERROR Procedure

Used to:

- Raise custom error messages
- Use error numbers from **-20000 to -20999**

Syntax:

```
sql

RAISE_APPLICATION_ERROR(-20001, 'Custom message');
```

📌 **MCQ Favorite**:

Error numbers must be between **-20000 and -20999**

---

17. Summary

You learned:

- Exception handling structure
- Types of exceptions
- Trapping predefined & non-predefined errors
- User-defined exceptions
- Error propagation
- `RAISE_APPLICATION_ERROR`

---

18. Practice Overview

Practice includes:

- Identifying exceptions
- Writing handlers
- Output-based MCQs
- Error-cause mapping

ALL COMMON PL/SQL ERRORS (VERY IMPORTANT FOR EXAM)

SELECT & DATA ERRORS

Error	Cause
NO_DATA_FOUND	No row returned
TOO_MANY_ROWS	Multiple rows
INVALID_NUMBER	Wrong number format
VALUE_ERROR	Datatype/size mismatch

ARITHMETIC ERRORS

Error	Cause
ZERO_DIVIDE	Divide by zero

CONSTRAINT ERRORS

Error	Cause
DUP_VAL_ON_INDEX	Duplicate key

CURSOR ERRORS

Error	Cause
CURSOR_ALREADY_OPEN	Opening twice
INVALID_CURSOR	Illegal fetch/close

USER & SYSTEM ERRORS

Error	Cause
LOGIN_DENIED	Invalid login
STORAGE_ERROR	Memory issue
PROGRAM_ERROR	Internal PL/SQL error

OTHERS

Error	Cause
OTHERS	Any unhandled error

VERY IMPORTANT MCQ QUICK REVISION

✔️ Exception section comes after BEGIN
✔️ Predefined exceptions need no declaration
✔️ **`WHEN OTHERS`** must be last
✔️ SQLCODE = error number
✔️ SQLERRM = error message
✔️ User-defined exceptions use RAISE
✔️ **`RAISE_APPLICATION_ERROR`** range = -20000 to -20999

1-LINE EXAM MEMORY TRICK

Predefined → automatic
Non-predefined → PRAGMA
User-defined → RAISE

If you want next, I can:
✅ Create **40 MCQs with answers**
✅ Give **error-cause mapping chart (1 page)**
✅ Explain **nested exception blocks**
✅ Provide **exam-ready notes PDF format**

Just tell me 👍

> explain everything there is about sql functions

Below is a **complete, exam-ready explanation of EVERYTHING about SQL FUNCTIONS**, written in **very simple words**, covering **all types, rules, syntax, examples, differences, and MCQ traps**.
After this, you should be able to answer **any MCQ or theory question** on **SQL functions** confidently.

I'll explain this in **Oracle / PL-SQL exam perspective**, because that's clearly your syllabus.

SQL FUNCTIONS – COMPLETE GUIDE (EXAM ORIENTED)

1. What is a SQL Function?

A **SQL function**:

- Takes **input values**
- Performs an **operation**
- Returns **a single output value**

📌 **MCQ Tip**

A function always returns **one value

> explain everything there is about plsql functions

Below is a **complete, exam-ready explanation of EVERYTHING about PL/SQL FUNCTIONS**, explained in **very simple words**, with **syntax, rules, examples, differences, internal working, and MCQ traps**.
After this, you should be able to answer **any MCQ or theory question** on **PL/SQL functions** confidently.

PL/SQL FUNCTIONS – COMPLETE GUIDE

1. What is a PL/SQL Function?

A **PL/SQL function** is a **named PL/SQL block** that:

- Performs a specific task
- **Always returns exactly ONE value**
- Can be used inside **SQL statements** and **PL/SQL blocks**

📌 **MCQ GOLD RULE**

> **Function must return a value**

---

2. Function vs Procedure (VERY IMPORTANT)

Feature	Function	Procedure
Returns value	✅ Yes	❌ No
RETURN keyword	Mandatory	Optional
Used in SQL	✅ Yes	❌ No
OUT parameters	Not preferred	Common
Call syntax	Expression	Statement

📌 **MCQ Trap**

> Procedure **cannot** be used in SQL statements

---

3. Structure of a PL/SQL Function

Syntax

```sql
sql

CREATE [OR REPLACE] FUNCTION function_name
   (parameter_name datatype [mode])
RETURN datatype
IS
   -- declaration section
BEGIN
   -- executable section
   RETURN value;
EXCEPTION
   -- exception handling
END;
/
```

---

4. Parts of a PL/SQL Function

1. CREATE OR REPLACE

- Creates a new function
- Replaces existing one without dropping

2. Function Name

- Must follow naming rules
- Should be meaningful

3. Parameters

Mode	Meaning
IN	Input only (default)
OUT	Output only
IN OUT	Input + Output

📌 MCQ Trap

Functions mainly use **IN parameters**

4. RETURN Clause

- Defines return datatype
- Only **one RETURN datatype allowed**

5. Declaration Section

- Variables
- Constants
- Cursors
- Exceptions

6. BEGIN–END Block

- Contains logic
- Must include at least **one RETURN statement**

7. EXCEPTION Section

- Optional
- Handles runtime errors

5. RETURN Statement (VERY IMPORTANT)

Rules:

✔️ Must return **same datatype** as RETURN clause
✔️ Execution stops after RETURN
✔️ Multiple RETURN statements allowed (but only one executed)

Example:

```
sql

RETURN salary * 0.1;
```

📌 **MCQ Trap**

> RETURN ends function execution immediately

---

6. Simple Function Example

```
sql

CREATE OR REPLACE FUNCTION get_bonus(
   p_salary NUMBER
)
RETURN NUMBER
IS
BEGIN
   RETURN p_salary * 0.10;
END;
/
```

---

7. Calling a Function

In PL/SQL Block

```
sql

v_bonus := get_bonus(50000);
```

In SQL Statement (⭐ IMPORTANT)

```
sql

SELECT get_bonus(salary) FROM employees;
```

📌 **MCQ GOLD**

> Functions can be used in SELECT, WHERE, HAVING

---

8. Function Determinism

A function is **deterministic** if:

- Same input → same output always

Syntax

```
sql

CREATE FUNCTION f_name
RETURN NUMBER DETERMINISTIC
```

📌 Used for **function-based indexes**

---

9. Functions and SQL Restrictions (VERY IMPORTANT)

When a function is used in SQL, it **CANNOT**:

❌ Perform DML (INSERT, UPDATE, DELETE)
❌ Use COMMIT / ROLLBACK
❌ Use non-deterministic operations

📌 **MCQ Trap**

> Functions used in SQL must be **pure**

---

10. Function Purity Levels

Purity Level	Meaning
WNDS	Writes No Database State
RNDS	Reads No Database State
WNPS	Writes No Package State
RNPS	Reads No Package State

📌 Mostly theoretical MCQs

---

11. Exception Handling in Functions

```
sql

EXCEPTION
   WHEN NO_DATA_FOUND THEN
      RETURN 0;
END;
```

📌 **MCQ Trap**

> Function must return a value even in exception

---

12. Returning Complex Datatypes

Functions can return:

- Scalar datatype
- Record
- Collection

Example (Return Record)

```
sql

RETURN v_emp;
```

---

13. Functions with Default Parameters

```
sql

CREATE FUNCTION tax(
   p_salary NUMBER,
   p_rate NUMBER DEFAULT 10
)
RETURN NUMBER
```

📌 Default parameters must be **at the end**

---

14. Overloading Functions

Multiple functions with:

- Same name
- Different parameter list

📌 Used inside **packages**

---

15. Stored vs Local Functions

Type	Scope
Stored Function	Database level
Local Function	Inside block

---

16. Local Function Example

```sql
DECLARE
   FUNCTION square(n NUMBER) RETURN NUMBER IS
   BEGIN
      RETURN n*n;
   END;
BEGIN
   DBMS_OUTPUT.PUT_LINE(square(5));
END;
```

17. Recursive Functions

Function calling itself.

Example:

```sql
FUNCTION fact(n NUMBER) RETURN NUMBER IS
BEGIN
   IF n = 1 THEN
      RETURN 1;
   ELSE
      RETURN n * fact(n-1);
   END IF;
END;
```

📌 Must have **base condition**

18. Difference: SQL vs PL/SQL Functions

SQL Function	PL/SQL Function
Built-in	User-defined
No exception block	Has exception block
Fixed behavior	Custom logic

19. Common Errors in Functions (MCQ IMPORTANT)

Error	Cause
Function returned without value	Missing RETURN
ORA-06503	No RETURN executed
ORA-06512	Stack trace
INVALID NUMBER	Wrong datatype

20. Advantages of Functions

✔️ Code reuse
✔️ Modular programming
✔️ Readability
✔️ Can be used in SQL

---

21. Summary

✔️ Function returns exactly one value
✔️ RETURN clause mandatory
✔️ Can be called from SQL & PL/SQL
✔️ Must follow purity rules
✔️ Supports exception handling

VERY IMPORTANT MCQ QUICK REVISION

✔️ Function must return a value
✔️ RETURN datatype defined in header
✔️ Functions can be used in SELECT
✔️ Procedures cannot be used in SQL
✔️ RETURN ends execution
✔️ Exception block optional

1-LINE EXAM MEMORY TRICK

> **Function = returns value + usable in SQL**

If you want next, I can:
✅ Create **MCQs with answers**
✅ Explain **functions vs procedures vs packages**
✅ Give **output-based questions**
✅ Provide **1-page exam notes**

Just tell me 👍

> explain everything about procedures

Below is a **complete, exam-ready explanation of EVERYTHING about PL/SQL PROCEDURES**, explained in **very simple words**, with **syntax, rules, examples, differences, internal behavior, errors, and MCQ traps**.
After this, you should be able to answer **any MCQ or theory question** on **procedures** confidently.

PL/SQL PROCEDURES – COMPLETE GUIDE

1. What is a PL/SQL Procedure?

A **PL/SQL procedure** is a **named PL/SQL block** that:

- Performs a task
- **Does NOT return a value directly**
- Can return values using **OUT / IN OUT parameters**
- Is mainly used for **actions (DML operations)**

📌 MCQ GOLD RULE

> **Procedure does not return a value using RETURN**

2. Procedure vs Function (VERY IMPORTANT)

Feature	Procedure	Function
Returns value	❌ No	✅ Yes
RETURN keyword	Optional (exit only)	Mandatory
Used in SQL	❌ No	✅ Yes
OUT parameters	✅ Yes	❌ Not preferred
Main use	Action / DML	Calculation

📌 MCQ Trap

> Procedures **cannot** be used in SELECT statements

3. Structure of a PL/SQL Procedure

Syntax

```
sql

CREATE [OR REPLACE] PROCEDURE procedure_name
   (parameter_name datatype [mode])
IS
   -- declaration section
BEGIN
   -- executable section
EXCEPTION
   -- exception handling
END;
/
```

4. Parts of a Procedure

1. CREATE OR REPLACE

- Creates or replaces procedure

---

2. Procedure Name

- Follows naming rules
- Must be unique in schema

---

3. Parameters

Parameter Modes

Mode	Meaning
IN	Input only (default)
OUT	Output only
IN OUT	Input + Output

📌 MCQ Trap

> OUT parameter value must be assigned inside procedure

---

4. Declaration Section

- Variables
- Constants
- Cursors
- Exceptions

---

5. BEGIN–END Block

- Contains logic
- DML statements allowed

---

6. EXCEPTION Section

- Optional
- Handles runtime errors

---

5. Simple Procedure Example

```sql
CREATE OR REPLACE PROCEDURE raise_salary(
   p_empid IN NUMBER,
   p_amount IN NUMBER
)
IS
BEGIN
   UPDATE employees
   SET salary = salary + p_amount
   WHERE employee_id = p_empid;

   COMMIT;
END;
/
```

📌 **MCQ Tip**

Procedures **can contain COMMIT / ROLLBACK**

---

6. Calling a Procedure

1. From PL/SQL Block

```sql
BEGIN
   raise_salary(101, 1000);
END;
```

---

2. Using EXEC

```sql
EXEC raise_salary(101, 1000);
```

---

3. Calling with OUT Parameter

```sql
DECLARE
   v_bonus NUMBER;
BEGIN
   get_bonus(50000, v_bonus);
END;
```

---

7. IN, OUT, IN OUT Parameters (IMPORTANT)

IN Parameter

- Default

- Read-only

OUT Parameter

- Used to return value
- Initial value ignored

IN OUT Parameter

- Input + output

📌 MCQ Trap

OUT parameter **cannot be read before assignment**

---

8. RETURN Statement in Procedure

- RETURN is **optional**
- Used to **exit procedure**, not return value

```sql
RETURN;
```

📌 MCQ Favorite

RETURN in procedure ≠ return value

---

9. Exception Handling in Procedures

```sql
EXCEPTION
   WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('No record');
END;
```

📌 MCQ Tip

Procedure may end without handling exception → propagated

---

10. Procedures with Default Parameters

```sql
PROCEDURE calc_tax(
   p_salary NUMBER,
   p_rate NUMBER DEFAULT 10
)
```

📌 Default parameters must be **last**

11. Stored vs Local Procedures

Type	Scope
Stored Procedure	Database level
Local Procedure	Inside block

12. Local Procedure Example

```sql
sql

DECLARE
   PROCEDURE greet IS
   BEGIN
      DBMS_OUTPUT.PUT_LINE('Hello');
   END;
BEGIN
   greet;
END;
```

13. Recursive Procedures

Procedure calling itself.

📌 Must have exit condition.

14. Procedures and Transactions

✔️ Procedures can perform:

- INSERT
- UPDATE
- DELETE
- COMMIT
- ROLLBACK

📌 **MCQ Trap**

Functions cannot COMMIT; procedures can

15. Procedures with Cursors

```sql
PROCEDURE list_emp IS
   CURSOR c IS SELECT * FROM employees;
BEGIN
   FOR r IN c LOOP
      DBMS_OUTPUT.PUT_LINE(r.employee_id);
   END LOOP;
END;
```

16. Procedures vs Packages

Procedures are often grouped inside **packages**.

Benefits:

- Encapsulation
- Overloading
- Security

17. Common Errors in Procedures (MCQ IMPORTANT)

Error	Cause
PLS-00306	Wrong parameters
ORA-06550	Syntax error
ORA-06512	Line number trace
NO_DATA_FOUND	SELECT INTO no row

18. Advantages of Procedures

✔️ Reusability
✔️ Modularity
✔️ Performance (stored once)
✔️ Security (grant EXECUTE)

19. Summary

✔️ Procedure does not return value
✔️ OUT parameters return data
✔️ Can perform DML & transactions
✔️ Cannot be used in SQL statements
✔️ RETURN only exits procedure

VERY IMPORTANT MCQ QUICK REVISION

✔️ Procedure ≠ function
✔️ OUT parameters must be assigned
✔️ Procedures allow COMMIT
✔️ RETURN does not return value
✔️ EXEC used to call procedure

1-LINE EXAM MEMORY TRICK

> **Procedure = action without return value**

If you want next, I can:
✅ Create **MCQs with answers**
✅ Compare **procedure vs function vs trigger**
✅ Explain **packages in full detail**
✅ Give **real-exam style questions**

Just tell me 👍

> explain everything about triggers and it's types if there is

Below is a **complete, exam-ready explanation of EVERYTHING about PL/SQL TRIGGERS**, explained in **very simple words**, with **definitions, syntax, types, timing, levels, examples, restrictions, errors, and MCQ traps**.
After this, you should be able to answer **any MCQ or theory question** on **triggers** confidently.

PL/SQL TRIGGERS – COMPLETE GUIDE

---

1. What is a Trigger?

A **trigger** is a **special PL/SQL block** that:

- Executes **automatically**
- Fires when a **specific event** occurs
- Is attached to a **table, view, schema, or database**

📌 **MCQ GOLD RULE**

> **Trigger fires automatically — it cannot be called explicitly**

---

2. Why Triggers are Used

Triggers are used to:

- Enforce **business rules**
- Maintain **audit logs**
- Validate data
- Prevent invalid operations
- Perform automatic actions

---

3. Trigger vs Procedure vs Function (IMPORTANT)

Feature	Trigger	Procedure	Function
Called explicitly	❌ No	✅ Yes	✅ Yes
Fired automatically	✅ Yes	❌ No	❌ No
Attached to table	✅ Yes	❌ No	❌ No
Returns value	❌ No	❌ No	✅ Yes
Uses :OLD / :NEW	✅ Yes	❌ No	❌ No

📌 **MCQ Trap**

> Triggers cannot be executed using EXEC

---

4. Trigger Syntax

```sql

CREATE [OR REPLACE] TRIGGER trigger_name
{BEFORE | AFTER | INSTEAD OF}
{INSERT | UPDATE | DELETE}
ON table_name
[FOR EACH ROW]
BEGIN
   -- trigger body
END;
/
```

---

5. Trigger Firing Events (DML Triggers)

Triggers can fire on:

Event	Meaning
INSERT	Row insertion
UPDATE	Row modification
DELETE	Row deletion

📌 **MCQ Tip**

DML triggers fire on INSERT, UPDATE, DELETE only

---

6. Trigger Timing (WHEN Trigger Fires)

1. BEFORE Trigger

- Fires **before** DML operation
- Used for **validation**

2. AFTER Trigger

- Fires **after** DML operation
- Used for **logging**

3. INSTEAD OF Trigger

- Used on **views**
- Replaces DML operation

📌 **MCQ GOLD**

INSTEAD OF triggers are used only on **views**

---

7. Trigger Levels

1. Statement-Level Trigger

- Fires **once per SQL statement**
- No `FOR EACH ROW`

2. Row-Level Trigger

- Fires **once per affected row**
- Uses `FOR EACH ROW`

📌 **MCQ Trap**

`FOR EACH ROW` → row-level trigger

---

8. OLD and NEW Pseudorecords (VERY IMPORTANT)

Used only in **row-level triggers**.

Event	:OLD	:NEW
INSERT	❌	✅
UPDATE	✅	✅
DELETE	✅	❌

Example

```sql
:NEW.salary := :OLD.salary + 1000;
```

📌 **MCQ Favorite**

:OLD is previous value, :NEW is new value

---

9. Types of Triggers (EXAM FAVORITE)

A. DML Triggers

- INSERT / UPDATE / DELETE
- Table or view

B. Row-Level Triggers

- **`FOR EACH ROW`**
- Use :OLD / :NEW

C. Statement-Level Triggers

- No **`FOR EACH ROW`**
- Executes once

D. INSTEAD OF Triggers

- On views only
- Handle complex view updates

E. Compound Triggers (Advanced / Theory)

- Combine multiple timings
- Improve performance

F. DDL Triggers

- Fire on CREATE, ALTER, DROP
- Schema or database level

```
sql

AFTER CREATE ON SCHEMA
```

G. Database Event Triggers

- Fire on:
  - LOGON
  - LOGOFF
  - STARTUP
  - SHUTDOWN

📌 MCQ Tip

LOGON triggers are database-level

---

10. Trigger Example (Row-Level BEFORE INSERT)

```
sql

CREATE OR REPLACE TRIGGER trg_salary_check
BEFORE INSERT ON employees
FOR EACH ROW
BEGIN
   IF :NEW.salary < 10000 THEN
      RAISE_APPLICATION_ERROR(-20001, 'Salary too low');
   END IF;
END;
/
```

---

11. WHEN Clause in Triggers

Used to apply condition.

```
sql

FOR EACH ROW
WHEN (:NEW.salary > 50000)
```

📌 MCQ Trap

WHEN clause cannot use PL/SQL variables

---

12. Mutating Table Error (VERY IMPORTANT)

Occurs when:

- Trigger tries to read/write same table

📌 **MCQ Favorite**

ORA-04091 = mutating table error

---

13. Restrictions on Triggers (EXAM IMPORTANT)

❌ Cannot COMMIT / ROLLBACK in DML trigger
❌ Cannot call procedure with COMMIT
❌ Cannot use :OLD/:NEW in statement trigger
❌ Cannot pass parameters to triggers

---

14. Enabling / Disabling Triggers

```sql
sql

ALTER TRIGGER trigger_name DISABLE;
ALTER TRIGGER trigger_name ENABLE;
```

---

15. Dropping a Trigger

```sql
sql

DROP TRIGGER trigger_name;
```

---

16. Errors Related to Triggers

Error	Cause
ORA-04098	Invalid trigger
ORA-04091	Mutating table
ORA-04092	COMMIT in trigger
ORA-06512	Error trace

---

17. Advantages of Triggers

✔️ Automatic execution
✔️ Centralized business logic
✔️ Data integrity
✔️ Security enforcement

---

18. Disadvantages of Triggers

❌ Hard to debug
❌ Performance overhead
❌ Hidden logic

---

19. Summary

✔️ Trigger fires automatically
✔️ Cannot be called explicitly
✔️ Can be BEFORE / AFTER / INSTEAD OF
✔️ Can be row-level or statement-level
✔️ Uses :OLD / :NEW
✔️ Cannot COMMIT

VERY IMPORTANT MCQ QUICK REVISION

✔️ Trigger ≠ procedure
✔️ INSTEAD OF trigger → view
✔️ FOR EACH ROW → row-level
✔️ :OLD / :NEW only in row triggers
✔️ No COMMIT in triggers
✔️ Mutating table error = ORA-04091

1-LINE EXAM MEMORY TRICK

> **Trigger = automatic action on event**

If you want next, I can:
✅ Create **40 MCQs with answers**
✅ Compare **trigger vs procedure vs constraint**
✅ Explain **compound triggers**
✅ Give **real-exam output questions**

Just tell me 👍